

Minggu 9 : Manajemen Memory di Linux

Tujuan Praktikum Manajemen Memory:

Mahasiswa mampu:

- Memahami konsep **manajemen memori utama**
- Menggunakan **perintah Linux untuk monitoring memory**
- Menganalisis **paging, virtual memory, dan swapping**
- Memahami **fragmentasi dan proteksi memory**

Dasar Teori

Memori utama (RAM) adalah tempat penyimpanan sementara yang digunakan CPU untuk menjalankan program.

Fungsi utama:

- Menyimpan program yang sedang dijalankan
- Menyimpan data yang sedang diproses
- Menyediakan akses cepat bagi CPU dibandingkan penyimpanan sekunder (hard disk)

Paging adalah teknik membagi memori menjadi bagian kecil dengan ukuran tetap.

Konsep:

- Page → potongan dari program
- Frame → potongan dari memori fisik

Cara kerja:

- Program dipecah menjadi page
- Memori dibagi menjadi frame
- OS mencocokkan page ke frame

Kelebihan:

- Menghindari fragmentasi eksternal
- Pengelolaan memori lebih fleksibel

Virtual memory adalah teknik yang membuat seolah-olah memori lebih besar dari kapasitas sebenarnya.

Cara kerja:

- Data yang tidak aktif disimpan di disk (swap space)
- Data aktif tetap di RAM

Manfaat:

- Bisa menjalankan program besar meskipun RAM terbatas
- Mendukung multitasking lebih baik

Swapping adalah proses memindahkan data antara RAM dan disk.

Jenis:

- Swap out → dari RAM ke disk
- Swap in → dari disk ke RAM

Tujuan:

- Membebaskan ruang RAM
- Menjalankan lebih banyak proses

Fragmentasi adalah kondisi memori tidak digunakan secara optimal.

Jenis:

1. Fragmentasi internal
 - Sisa ruang dalam blok yang tidak terpakai
2. Fragmentasi eksternal
 - Banyak ruang kosong kecil yang terpisah-pisah

Proteksi memori adalah mekanisme untuk menjaga agar proses tidak saling mengganggu.

Tujuan:

- Mencegah satu proses mengakses memori proses lain
- Menjaga stabilitas sistem
- Menghindari crash atau error

1. PERINTAH LINUX TERKAIT MEMORY

Tujuan : mengetahui kondisi memory sistem secara real-time

Perintah yang Digunakan

1. Melihat penggunaan RAM

```
$ free -h
```

Contoh luaran:

```
$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	7.6Gi	2.3Gi	1.2Gi	200Mi	4.1Gi	4.8Gi
Swap:	2.0Gi	0.5Gi	1.5Gi			

2. Detail memory sistem

```
$ cat /proc/meminfo
```

Contoh luaran:

```
$ cat /proc/meminfo
MemTotal:      8164320 kB
MemFree:       1023456 kB
MemAvailable:  4215672 kB
Buffers:       234512 kB
Cached:        3124560 kB
SwapCached:    65432 kB
Active:        2987654 kB
Inactive:      1876543 kB
Active(anon):  1456789 kB
Inactive(anon): 765432 kB
Active(file):  1530865 kB
Inactive(file): 1111111 kB
Unevictable:   2048 kB
Mlocked:       2048 kB
SwapTotal:     2097148 kB
SwapFree:      1567890 kB
```

Cara memahami informasi penting di atas:

a. Memory Utama :

- **MemTotal** → Total RAM (≈ 8 GB)
- **MemFree** → RAM kosong
- **MemAvailable** → RAM yang masih bisa dipakai tanpa swap  (paling penting)

b. Cache & Buffer

- **Buffers** → buffer I/O
- **Cached** → cache file (mempercepat akses disk)

c. Aktivitas Memory

- **Active** → sedang dipakai
- **Inactive** → jarang dipakai (bisa di-reclaim)

d. Swap

- **SwapTotal** → total swap
- **SwapFree** → swap tersisa

3. Monitoring proses

\$ top atau \$ htop

Contoh output top:

```
top - 10:15:32 up 2:35, 2 users, load average: 0.15, 0.10, 0.05
Tasks: 205 total, 1 running, 204 sleeping, 0 stopped, 0 zombie
%Cpu(s): 5.2 us, 2.1 sy, 0.0 ni, 92.3 id, 0.2 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 7972.0 total, 1200.0 free, 2300.0 used, 4472.0 buff/cache
MiB Swap: 2048.0 total, 1500.0 free, 548.0 used. 4800.0 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1023	user	20	0	21568	3456	2100	S	0.3	0.0	0:00.12	bash
2045	user	20	0	250000	50000	10000	S	1.5	0.6	0:05.30	firefox
3100	user	20	0	150000	30000	8000	S	0.7	0.3	0:02.10	code

Informasi Penting :

- **load average** → beban sistem
- **Tasks** → jumlah proses
- **%Cpu(s)** → penggunaan CPU
- **Mem** → penggunaan RAM
- **Swap** → penggunaan swap

4. Informasi proses tertentu

\$ ps aux | grep <nama_proses>

Contoh output perintah : \$ ps aux | grep bash

user	1023	0.0	0.1	21568	3456	pts/0	Ss	10:00	0:00	bash
user	2045	0.0	0.0	14224	1024	pts/1	S+	10:05	0:00	grep bash

Penjelasan tiap kolom output perintah di atas (dari kiri ke kanan) :

User = pemilik proses, 1023 = process ID, 0.0 = % penggunaan CPU, 0.1 = % penggunaan memory, 21568 = virtual memory (kbytes), 3456 = physical memory (kbytes), pts/0 = terminal, Ss = status proses, 10:00 = waktu mulai, 0:00 = total waktu CPU, bash = nama perintah.

5. Penggunaan memory per proses

\$ pmap <PID>

Contoh luaran perintah : \$ pmap 2025 (2045= proses ID)

```
2045:  /usr/lib/firefox/firefox
000055555554000    1024K r-x-- firefox
00005555555654000    256K r---- firefox
00005555555694000    128K rw--- firefox
000055555556b4000     64K rw--- [ anon ]
00007ffff7a0d000   2048K r-x-- libc-2.31.so
00007ffff7c0d000   1024K ----- libc-2.31.so
00007ffff7d0d000     64K r---- libc-2.31.so
00007ffff7d1d000     32K rw--- libc-2.31.so
00007ffff7f00000   4096K rw--- [ anon ]
00007ffff7ffde000    132K rw--- [ stack ]
00007ffff7ffff000     4K r-x-- [ anon ]
total                8836K
```

Keterangan luaran (dari kolom kiri sd kanan):

Alamat virtual, ukuran memory, hak akses, nama program yang sedang berjalan.

6. Statistik memory

\$ vmstat 1 --- perintah ini menampilkan statistik setiap 1 detik.

Contoh luaran :

```
procs -----memory----- --swap-- -----io----- -system-- -----cpu-----
r  b  swpd  free  buff  cache  si  so  bi  bo  in  cs  us  sy  id  wa  st
1  0  524288 1200000 200000 3500000 10 20 15 30 120 200 5 2 92 1 0
0  0  524288 1180000 200500 3510000 0 5 0 10 110 180 3 1 95 1 0
2  1  530000 1100000 201000 3400000 20 15 50 60 200 300 10 5 80 5 0
```

Keterangan per kolom:

r = jumlah proses yang siap jalan (running)

b = proses yang menunggu I/O (blocked)

swpd = memory yang digunakan di swap

free = memory kosong

buff = buffer, cache = cache

si = swap in (dari disk ke RAM)

so = swap out (dari RAM ke disk)

bi = block in (disk read)

bo = block out (disk write)

Penting: jika si dan so besar → sistem sedang **kekurangan RAM**

7. Cache & buffer

Perintah berikut digunakan untuk membersihkan cache memory di Linux:

```
$ sync; echo 3 > /proc/sys/vm/drop_caches
```

```
$ sync; echo 3 > /proc/sys/vm/drop_caches
```

```
bash: /proc/sys/vm/drop_caches: Permission denied -- perlu hak root
```

```
$ sudo sh -c "sync; echo 3 > /proc/sys/vm/drop_caches"
```

Perintah di atas tidak menghasilkan luaran, tetapi cache sudah dibersihkan.

Percobaan 1.

1. Jalankan free -h, catat total RAM dan free memory
2. Jalankan top, cari proses dengan memory terbesar
3. Gunakan pmap pada salah satu proses

2. PAGING & PAGE TABLE

Tujuan : memahami bagaimana memory dibagi dalam bentuk page

1. Melihat ukuran page

```
$ getconf PAGE_SIZE
```

```
$ getconf PAGE_SIZE
```

```
4096
```

Penjelasan singkat:

- 4096 berarti ukuran halaman (page size) adalah 4096 byte (4 KB)
- Ini adalah ukuran umum pada banyak sistem Linux modern

Artinya:

- Memori dibagi dalam blok-blok kecil berukuran tetap 4 KB
- Digunakan dalam mekanisme paging dan virtual memory
- Pada beberapa sistem, nilai bisa berbeda, misalnya: 8192 atau 16384, tergantung arsitektur hardware dan konfigurasi OS

2. Informasi page system

```
$ cat /proc/self/maps
```

```
$ cat /proc/self/maps
55d3c2f3d000-55d3c2f3f000 r--p 00000000 08:01 131082 /usr/bin/cat
55d3c2f3f000-55d3c2f43000 r-xp 00002000 08:01 131082 /usr/bin/cat
55d3c2f43000-55d3c2f45000 r--p 00006000 08:01 131082 /usr/bin/cat
55d3c2f45000-55d3c2f46000 r--p 00007000 08:01 131082 /usr/bin/cat
55d3c2f46000-55d3c2f47000 rw-p 00008000 08:01 131082 /usr/bin/cat
55d3c3b8b000-55d3c3bac000 rw-p 00000000 00:00 0 [heap]
7f8c2c000000-7f8c2c021000 rw-p 00000000 00:00 0
7f8c2c021000-7f8c30000000 ---p 00000000 00:00 0
7f8c31c00000-7f8c31c21000 rw-p 00000000 00:00 0
7f8c31c21000-7f8c35c00000 ---p 00000000 00:00 0
7f8c37ffd000-7f8c3801e000 rw-p 00000000 00:00 0 [stack]
7f8c3805a000-7f8c3805c000 r--p 00000000 00:00 0 [vvar]
7f8c3805c000-7f8c3805e000 r-xp 00000000 00:00 0 [vdso]
ffffffffffff600000-ffffffffffff601000 --xp 00000000 00:00 0 [vsyscall]
```

Setiap baris menunjukkan **pemetaan memori (memory mapping)** dengan format:

[address_awal]-[address_akhir] [permission] [offset] [device] [inode] [path]

3. Statistik paging

\$ vmstat

```
$ vmstat
procs -----memory----- --swap-- -----io----- -system-- -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo    in   cs us sy id wa st
 1  0       0 245000 35000 120000    0    0     5     3   120  250 10  5 80  5  0
```

Proses (procs)

- **r** → jumlah proses yang siap dijalankan (running)
- **b** → proses yang sedang menunggu (blocked)

Memori (memory)

- **swpd** → jumlah memori yang digunakan sebagai swap (KB)
- **free** → memori bebas
- **buff** → buffer untuk I/O
- **cache** → cache file system

Swap

- **si (swap in)** → data dari disk ke RAM
- **so (swap out)** → data dari RAM ke disk

I/O

- **bi** → blok masuk dari disk
- **bo** → blok keluar ke disk

Sistem (system)

- **in** → interrupt per detik
- **cs** → context switch per detik

CPU (cpu)

- **us** → waktu CPU untuk user process
- **sy** → waktu untuk kernel
- **id** → idle (tidak digunakan)

- **wa** → menunggu I/O
- **st** → waktu “dicuri” (virtual machine)

Percobaan 2 :

1. Cek ukuran page
2. Analisis output /proc/self/maps
3. Identifikasi segment memory (heap, stack, dll)

3. PROTEKSI MEMORY

Tujuan : memahami bagaimana OS melindungi memory antar proses

Perintah

1. Lihat permission memory

```
$ cat /proc/<PID>/maps
```

(PID ganti dengan nomer proses yang ingin diamati)

Contoh output:

r-xp → read & execute

rw-p → read & write

2. Uji proteksi sederhana (C program)

```
#include <stdio.h>

int main() {
    int *p = NULL;
    *p = 10; // akses ilegal
    return 0;
}
```

Penjelasan Baris Penting

- ♦ `int *p = NULL;`

- Mendeklarasikan pointer p yang menunjuk ke alamat **NULL (0)**
- Artinya:
 - p **tidak menunjuk ke lokasi memori yang valid**
 - NULL = “tidak ada alamat”
 - Sistem operasi **melarang akses ke alamat ini**
 - Akan terjadi **akses ilegal (invalid memory access)**
 - Program akan crash dengan error:

Segmentation fault (core dumped)

Compile & run:

```
$ gcc test.c -o test
$ ./test
```

Percobaan 3:

1. Jalankan program di atas
2. Jelaskan kenapa terjadi **segmentation fault**

4. SWAPPING

Tujuan: memahami penggunaan swap memory

Perintah yang berhubungan dengan proses swapping:

1. Cek swap

```
$ swapon -show
```

```
$ swapon --show
NAME      TYPE SIZE   USED PRIO
/swapfile file  2G     0B   -2
```

Penjelasan Kolom

- **NAME** → lokasi swap (file atau partisi)
- **TYPE** → jenis swap (file atau partition)
- **SIZE** → ukuran total swap
- **USED** → jumlah swap yang sedang digunakan
- **PRIO** → prioritas swap (semakin tinggi → lebih diprioritaskan)

2. Lihat swap usage

\$ free -h

```
$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	7.7G	2.1G	1.5G	200M	4.1G	5.1G
Swap:	2.0G	0B	2.0G			

Penjelasan Kolom

♦ Bagian Memori (RAM)

- total → total RAM yang tersedia
- used → RAM yang sedang digunakan
- free → RAM yang benar-benar kosong
- shared → memori yang digunakan bersama antar proses
- buff/cache → memori untuk buffer & cache (bisa dibebaskan jika diperlukan)
- available → estimasi RAM yang masih bisa dipakai aplikasi baru

♦ Bagian Swap

- total → total swap
- used → swap yang sedang digunakan
- free → swap yang masih tersedia

3. Monitoring swap

```
$ vmstat
```

Percobaan 5:

1. Jalankan program berat (misal membuka banyak tab browser)
2. Amati perubahan swap
3. Catat kapan swap mulai digunakan

5. VIRTUAL MEMORY

Tujuan: memahami konsep virtual memory di Linux

Perintah

1. Cek virtual memory

```
$ ulimit -a
```

```
$ ulimit -a
core file size          (blocks, -c) 0
data seg size           (kbytes, -d) unlimited
scheduling priority     (-e) 0
file size               (blocks, -f) unlimited
pending signals         (-i) 257239
max locked memory       (kbytes, -l) 64
max memory size         (kbytes, -m) unlimited
open files              (-n) 1024
pipe size               (512 bytes, -p) 8
POSIX message queues    (bytes, -q) 819200
real-time priority      (-r) 0
stack size              (kbytes, -s) 8192
cpu time                (seconds, -t) unlimited
max user processes      (-u) 127432
virtual memory          (kbytes, -v) unlimited
file locks              (-x) unlimited
```

2. Statistik VM

```
$ vmstat
```

```
$ vmstat
procs -----memory----- --swap-- ----io---- -system-- -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo    in   cs us sy id wa st
 2   0       0 180000  25000 150000    0    0   10    5   200  300 15 10 70  5  0
```

3. Overcommit memory

```
$ cat /proc/sys/vm/overcommit_memory
```

```
$ cat /proc/sys/vm/overcommit_memory
```

```
0
```

Arti Nilai overcommit_memory

Parameter ini mengatur **kebijakan alokasi memori (memory overcommit)** di Linux.

Nilai yang mungkin:

◆ 0 (default)

- Kernel **mengizinkan overcommit secara heuristik**
- Artinya:
 - Program boleh meminta memori lebih besar dari RAM + swap
 - Tapi kernel tetap melakukan perkiraan agar tidak berlebihan

✦ Ini adalah mode yang paling umum digunakan

◆ 1

- Kernel **selalu mengizinkan overcommit**
- Tidak peduli kapasitas memori

⚠ Risiko:

- Bisa menyebabkan **Out Of Memory (OOM Killer)**
- Program tiba-tiba dihentikan

◆ 2

- Kernel **tidak mengizinkan overcommit**
- Alokasi memori dibatasi ketat:

- Maksimum = RAM + swap (dengan batas tertentu)

Penjelasan

- Virtual memory memungkinkan:
 - Program lebih besar dari RAM
 - Isolasi proses
- Menggunakan:
 - Paging
 - Swapping

Percobaan 6.

1. Jalankan banyak program
2. Amati penggunaan memory vs swap
3. Analisis hubungan virtual memory dan swap

Analisis Percobaan:

Studi Kasus:

1. Jalankan beberapa aplikasi sekaligus
2. Monitor:
 - RAM usage
 - Swap usage
 - Proses terbesar

Pertanyaan Analisi :

1. Apa yang terjadi saat RAM penuh?
2. Kapan swap digunakan?
3. Apa dampak swapping terhadap performa?
4. Jelaskan hubungan:

- Paging
- Virtual Memory
- Swapping